

QUESTION NO: 2

- Write pseudocode for a producer-consumer system using counting semaphores and a bounded buffer.

PROBLEM UNDERSTANDING:

A Producer task generates data and puts it into a bounded buffer. A Consumer task takes the data from the buffer and processes it. If the buffer is full, the producer waits. If the buffer is empty, the consumer waits. Counting semaphores and a mutex are used for safe and efficient communication.

RTOS MECHANISM USED:

Counting Semaphores + Mutex

STRUCTURED PSEUDOCODE:

// GLOBAL DEFINITIONS

- DEFINE BUFFER
- DEFINE SEMAPHORE Empty
- DEFINE SEMAPHORE Full
- DEFINE MUTEX BufferMutex
- DEFINE CONSTANT BUFFER_SIZE

DATA STRUCTURE: BufferData

- data : INTEGER
- position : INTEGER

REAL-LIFE EXAMPLE:

Computer sends print jobs to a printer.

- Producer : Computer (sending jobs)
- Buffer : Printer Queue (limited size)
- Consumer : Printer (processing jobs)
- Empty : Free space in queue
- Full : Jobs waiting in queue
- Mutex : Protects the queue

PSEUDOCODE (Contd...)

BEGIN

// SYSTEM INITIALIZATION

```
CREATE BUFFER with size BUFFER_SIZE
CREATE SEMAPHORE Empty
INITIALIZE Empty = BUFFER_SIZE
CREATE SEMAPHORE Full
INITIALIZE Full = 0
CREATE MUTEX BufferMutex
INITIALIZE BufferMutex = 1
CREATE TASK ProducerTask WITH PRIORITY 2
CREATE TASK ConsumerTask WITH PRIORITY 2
START SCHEDULER
```

END

TASK: ProducerTask

BEGIN

DECLARE item

WHILE TRUE

```
GENERATE item // produce data
WAIT(Empty) // wait if buffer full
WAIT(BufferMutex) // lock buffer
INSERT item INTO BUFFER // add data
SIGNA(BufferMutex) // unlock buffer
SIGNA(Full) // data available
```

ENDWHILE

END

PSEUDOCODE (Contd...)

TASK: ConsumerTask

BEGIN

DECLARE item

WHILE TRUE

```
WAIT(Full) // wait if buffer empty
WAIT(BufferMutex) // lock buffer
REMOVE item FROM BUFFER // remove data
SIGNA(BufferMutex) // unlock buffer
SIGNA(Empty) // space available
PROCESS item // consume data
```

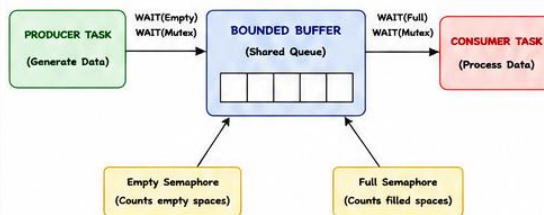
ENDWHILE

END

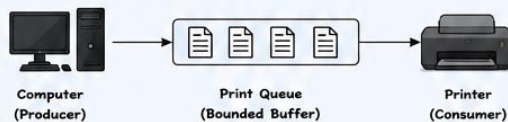
SEMAPHORE / MUTEX OPERATIONS SUMMARY

Operation	Performed By	Purpose
WAIT(Empty)	Producer	Waits if buffer is full
SIGNA(Empty)	Consumer	Informs empty space available
WAIT(Full)	Consumer	Waits if buffer is empty
SIGNA(Full)	Producer	Informs data available
WAIT(BufferMutex)	Producer/Consumer	Locks buffer for safe access
SIGNA(BufferMutex)	Producer/Consumer	Releases buffer

SYSTEM OVERVIEW

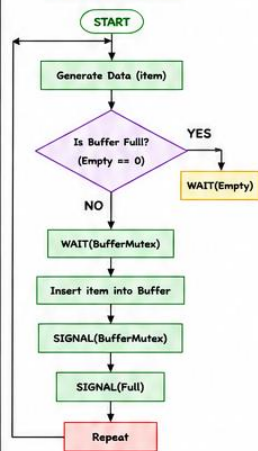


Real-Life Example: Printer System

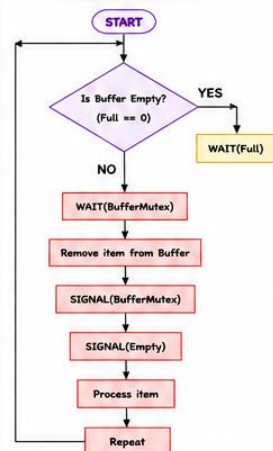


LOGICAL FLOW DIAGRAM

PRODUCER TASK FLOW



CONSUMER TASK FLOW



EXPLANATION:

The Producer task produces data and tries to insert it into the bounded buffer. If the buffer is full (no empty space), it waits on Empty semaphore. When space is available, it locks the buffer using BufferMutex, inserts the item, unlocks the buffer, and signals Full to indicate that data is available.

The Consumer task tries to remove data from the buffer. If the buffer is empty (no data), it waits on Full semaphore. When data is available, it locks the buffer, removes the item, unlocks the buffer, signals Empty to indicate that a space is free, and then processes the item.

This mechanism ensures safe access to the buffer and prevents data loss.

KEY POINTS:

- Empty semaphore counts available free spaces in the buffer.
- Full semaphore counts the number of data items in the buffer.
- Mutex ensures that only one task accesses the buffer at a time.
- Producer adds data; Consumer removes data.
- If buffer is full, producer waits.
- If buffer is empty, consumer waits.
- This method ensures safe and efficient task-to-task communication.

ASSUMPTIONS:

- Buffer size (BUFFER_SIZE) is predefined and fixed.
- Producer always generates valid data.
- Consumer can process any received data.
- Semaphore and mutex operations are atomic and provided by the RTOS.
- Scheduler manages task execution based on priority.

QUESTION NO: 3 – Multiple tasks accessing a shared register using a mutex with timeout.

PROBLEM UNDERSTANDING:

- Many tasks try to access a common shared register. A mutex is used to allow only one task at a time to access it. If a task cannot get the mutex in time, it gives up (timeout).

RTOS MECHANISM USED:

- Mutex with Timeout

STRUCTURED PSEUDOCODE:

// GLOBAL DEFINITIONS

- SHARED_REG = 0
- MUTEX
- TIMEOUT = 5 ticks.

DATA STRUCTURE:

- shared_reg : INTEGER
- mutex : MUTEX

PSEUDOCODE (Contd...)

TASK: Task (ID)

BEGIN

```
IF xSemaphoreTake(mutex, TIMEOUT) = SUCCESS THEN
    read(shared_reg) / write(shared_reg)
    xSemaphoreGive(mutex)
ELSE
    // Timeout - could not get mutex in time
    handle_timeout()
END IF
END
```

TASK: ConsumerTask

BEGIN

```
IF xSemaphoreTake(mutex, TIMEOUT) = SUCCESS THEN
    read(shared_reg)
    xSemaphoreGive(mutex)
ELSE
    handle_timeout()
END IF
END
```

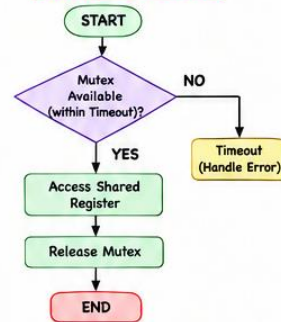
ASSUMPTIONS:

- Mutex protects the shared register.
- Timeout is fixed (e.g., 5 ticks).
- Tasks run under RTOS scheduler.

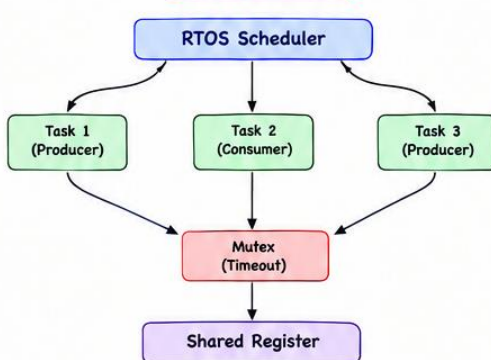
QUEUE OPERATIONS SUMMARY

Operation	Purpose
TAKE (mutex)	Lock the shared resource
GIVE (mutex)	Release the resource
TAKE with TIMEOUT	Wait for mutex, else timeout

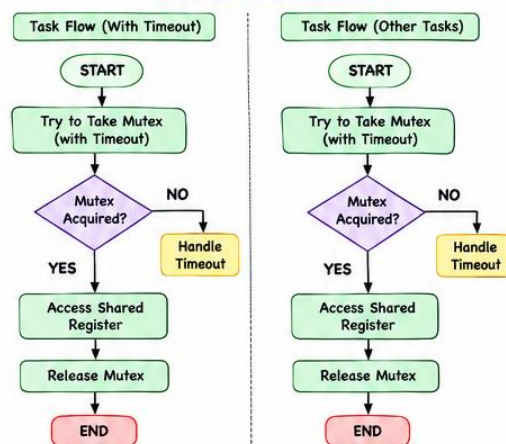
GLOBAL TASK FLOW



SYSTEM OVERVIEW



LOGICAL FLOW DIAGRAM



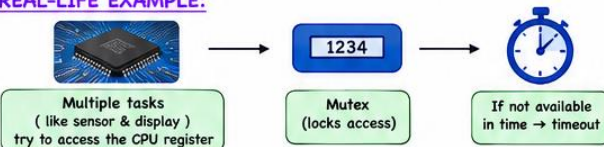
EXPLANATION:

- A mutex is used to lock the shared register so that only one task can access it at a time.
- When a task wants to access the register, it tries to take the mutex.
- If the mutex is not available within the given timeout, the task does not wait forever — it gives up and handles the timeout.
- This prevents deadlock and keeps the system responsive.

KEY POINTS:

- Only one task can access the shared register at a time.
- Mutex provides mutual exclusion.
- Timeout avoids waiting forever.
- Improves system reliability and responsiveness.
- Prevents resource contention.

REAL-LIFE EXAMPLE:



ASSUMPTIONS:

- Tasks try to access the shared register.
- Timeout value is configured.
- Mutex ensures mutual exclusion.
- RTOS scheduler manages task execution.

QUESTION NO: 4

- Write pseudocode showing priority inversion and priority inheritance using three tasks and a mutex.

PROBLEM UNDERSTANDING:

When a high priority task needs a resource (protected by a mutex) that is currently locked by a low priority task, a medium priority task may preempt the low priority task, causing the high priority task to wait longer. This is called priority inversion. Priority inheritance temporarily boosts the low priority task's priority to that of the high priority task, so that it can release the mutex quickly and unblock the high priority task.

RTOS MECHANISM USED:

Mutex with Priority Inheritance

STRUCTURED PSEUDOCODE:

```
// GLOBAL DEFINITIONS
- DEFINE MUTEX M (with Priority Inheritance)
```

REAL-LIFE EXAMPLE:

In an industrial control system

- Task L (Low) : Logger Task
- Task M (Medium) : UI Task
- Task H (High) : Control Task
- Mutex M : Shared Resource

PSEUDOCODE (Contd...)

TASK L (Low Priority)

```
BEGIN
WHILE TRUE
    WAIT(M)           // lock mutex
    DO_CRITICAL_WORK() // access shared resource
    SIGNAL(M)          // release mutex
    DELAY(10)
END WHILE
END
```

TASK H (High Priority)

```
BEGIN
WHILE TRUE
    WAIT(M)           // may block if L holds M
    DO_CRITICAL_WORK()
    SIGNAL(M)          // release mutex
    DELAY(10)
END WHILE
END
```

TASK M (Medium Priority)

```
BEGIN
WHILE TRUE
    DO_BACKGROUND_WORK() // non critical work
    DELAY(20)
END WHILE
END
```

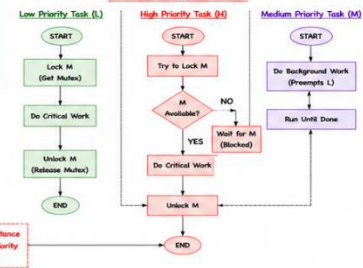
PRIORITY INHERITANCE BEHAVIOR:

- L tries to lock M, but L holds it.
- H is ready and preempts L.
- H tries to lock M, but M is held by L. H is blocked.
- M is ready and preempts L. (This is priority inversion)
- Priority inheritance raises L's priority to that of H.
- L completes work and releases M.
- L's priority returns to normal.
- H gets M and executes.

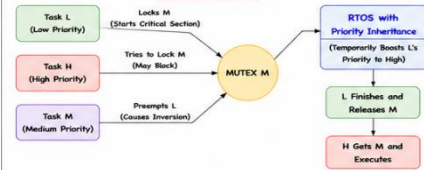
MUTEX OPERATIONS SUMMARY

Operation	Performed By	Purpose
WAIT(M)	Task	Lock mutex M
SIGNAL(M)	Task	Release mutex M
PRIORITY INHERITANCE	RTOS	Boost priority of mutex holder

LOGICAL FLOW DIAGRAM



SYSTEM OVERVIEW



TASK STATE TRANSITION (WITH PRIORITY INHERITANCE)

Step	Event	L (Low)	M (Med)	H (High)
1	L locks M	Running	Ready	Ready
2	H tries to lock M	Running	Ready	Blocked
3	M runs and preempts L (Priority Inversion)	Ready	Running	Blocked
4	Priority Inheritance (L boosted to High)	Running (Boosted)	Ready	Blocked
5	L releases M	Ready (Normal)	Ready	Ready
6	H locks M	Ready	Ready	Running

Real-Life Example: Industrial Control System



How it works:

The logger task (L) locks the configuration data to write logs. The control task (H) needs the data to send commands but gets blocked. The UI task (M) runs in between and delays L from finishing. Priority inheritance boosts L's priority so it can finish quickly and release the data, allowing H to run.

EXPLANATION:

Initially, low priority task L, locks the mutex M and starts its critical work. When high priority task H tries to lock M, it gets blocked. Medium priority task M runs and preempts L, causing priority inversion. With priority inheritance, RTOS temporarily boosts L's priority to high. L completes its work and releases M quickly. H gets M and executes. After that, L's priority returns to normal.

KEY POINTS:

- Priority inversion occurs when a low priority task blocks a high priority task.
- Priority inheritance temporarily boosts the low priority task.
- This allows the low task to finish and release the mutex quickly.
- High priority task can then proceed without long delay.
- This method ensures predictable timing in RTOS.

ASSUMPTIONS:

- Three tasks: L (Low), M (Medium), H (High).
- Mutex M uses Priority Inheritance protocol.
- Tasks are periodic or event-driven.
- Preemptive priority-based scheduling.
- Critical section is short.

QUESTION NO: 5

- Write pseudocode for writer and reader tasks safely sharing a common buffer using semaphores.

PROBLEM UNDERSTANDING:

Multiple readers can read the buffer. Writers need exclusive access to write. Semaphores are used to ensure mutual exclusion for writing and safe access for multiple readers.

RTOS MECHANISM USED:

Counting Semaphores (Mutex + Read Count)

STRUCTURED PSEUDOCODE:

```
// GLOBAL DEFINITIONS
- DEFINE SEM_MUTEX      Mutex = 1
- DEFINE SEM_WRITER      write = 1
- DEFINE SEMAPHORE      read_count = 0
- DEFINE BUFFER           BUFFER[]
```

REAL-LIFE EXAMPLE:

Database System

- Writers : Update database
- Readers : Read data
- Buffer : Database records

PSEUDOCODE (Contd...)

READER TASK

```
BEGIN
WAIT(Mutex)
read_count = read_count + 1
IF read_count == 1
    WAIT(write)
END IF
SIGNAL(Mutex)
READ from BUFFER
WAIT(Mutex)
read_count = read_count - 1
IF read_count == 0
    SIGNAL(write)
END IF
SIGNAL(Mutex)
END
```

WRITER TASK

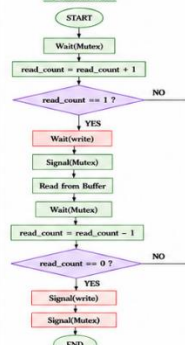
```
BEGIN
WAIT(write)
WRITE to BUFFER
SIGNAL(write)
END
```

SEMAPHORE OPERATIONS SUMMARY

Operation	Performed By	Purpose
WAIT(Mutex)	Reader / Writer	Protect read_count
SIGNAL(Mutex)	Reader / Writer	Release Mutex
WAIT(write)	Writer	Get write access
SIGNAL(write)	Writer / Last Reader	Release write access

LOGICAL FLOW DIAGRAM

READER FLOW



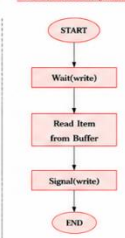
WRITER FLOW



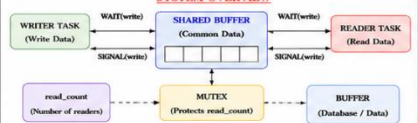
PRODUCER FLOW (Writer)



CONSUMER FLOW (Reader)



SYSTEM OVERVIEW



SEMAPHORE PURPOSE SUMMARY

Semaphore	Initial Value	Purpose
Mutex	1	Protects read_count variable
write	1	Ensures exclusive write access
read_count	0	Counts number of active readers

REAL-LIFE EXAMPLE: DATABASE SYSTEM



How it works:

Writers update the database (exclusive access required). Multiple readers can read simultaneously. Semaphores ensure safe and consistent access.

EXPLANATION:

If no reader is reading, a writer can write. If readers are reading, other readers can also read. But writers must wait until all readers finish. The first reader locks the write semaphore. The last reader releases it. This ensures writers get exclusive access and readers can read concurrently. This method prevents data corruption and ensures safe sharing.

KEY POINTS:

- Multiple readers can read simultaneously.
- Writers get exclusive access.
- First reader locks write semaphore.
- Last reader releases write semaphore.
- Mutex protects read_count variable.
- Prevents data inconsistency.
- Ensures efficient and fair access.

LOSUMATION:

- Protects critical section for writing.
- Allows concurrent reading.
- Ensures mutual exclusion for writers.
- Prevents data corruption.
- Supports multiple readers and one writer.
- Improves system performance and throughput.

ASSUMPTIONS:

- Mutex and semaphores are binary counting semaphores.
- read_count is integer and initialized to 0.
- Semaphore operations are atomic.
- Writers and readers are independent tasks.
- Scheduler handles task execution.

QUESTION NO: 6

- Write pseudocode for multiple producer and consumer tasks using semaphores safely.

PROBLEM UNDERSTANDING:

Multiple producers generate data and put it into a shared buffer. Multiple consumers take data from the buffer and process it. Semaphores are used to ensure mutual exclusion and correct counting of empty and full slots.

RTOS MECHANISM USED:

Counting Semaphores + Mutex

STRUCTURED PSEUDOCODE:

// GLOBAL DEFINITIONS

```
- DEFINE SEM_MUTEX      Mutex = 1
- DEFINE SEM_EMPTY      empty = N
- DEFINE SEM_FULL       full = 0
- DEFINE BUFFER          BUFFER[N]
- DEFINE IN, OUT         Integers
```

REAL-LIFE EXAMPLE:

Warehouse Inventory System
- Producers : Suppliers adding items
- Consumers : Stores requesting items
- Buffer : Inventory storage slots

PSEUDOCODE (Contd...)

PRODUCER TASK (Multiple)

```
BEGIN
  WHILE TRUE
    item = PRODUCE_ITEM() // generate item
    // wait if buffer full
    WAIT(empty)           // enter critical section
    BUFFER[in] = item      // put item in buffer
    in = (in + 1) % N      // circular increment
    SIGNAL(mutex)          // leave critical section
    SIGNAL(full)            // one more full slot
  END WHILE
END
```

CONSUMER TASK (Multiple)

```
BEGIN
  WHILE TRUE
    WAIT(full)             // wait if buffer empty
    WAIT(mutex)            // enter critical section
    item = BUFFER[out]     // take item from buffer
    // circular increment
    SIGNAL(mutex)          // leave critical section
    SIGNAL(empty)          // one more empty slot
    CONSUME_ITEM(item)    // process item
  END WHILE
END
```

SEMAPHORE OPERATIONS SUMMARY

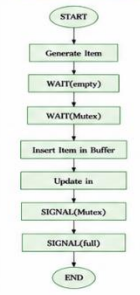
Operation	Performed By	Purpose
WAIT(empty)	Producer	Waits if buffer is full
WAIT(mutex)	Producer / Consumer	Provides mutual exclusion
SIGNAL(mutex)	Producer / Consumer	Releases critical section
SIGNAL(full)	Producer	Informs one more full slot
WAIT(full)	Consumer	Waits if buffer is empty
SIGNAL(empty)	Consumer	Informs one more empty slot

SEMAPHORE INITIAL VALUES

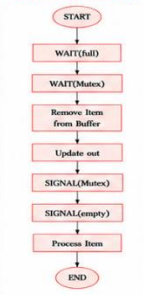
Semaphore	Initial Value	Purpose
Mutex	1	Ensures mutual exclusion
empty	N	Counts empty slots in buffer
full	0	Counts full slots in buffer

LOGICAL FLOW DIAGRAM

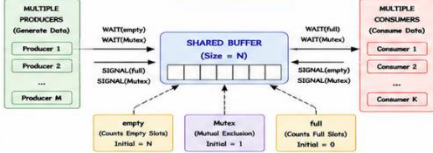
PRODUCER FLOW (Multiple)



CONSUMER FLOW (Multiple)



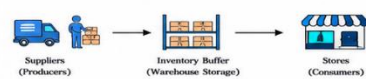
SYSTEM OVERVIEW



BUFFER STATE EXAMPLE

Step	Action	in	out	empty	full	Buffer Content
Initial State	-	0	0	N	0	[- - -]
P1 Producers	Insert Item A	1	0	N-1	1	[A - -]
P2 Producers	Insert Item B	2	0	N-2	2	[A B -]
C1 Consumers	Remove Item A	2	1	N-1	1	[- B -]
P3 Producers	Insert Item C	3	1	N-2	2	[- B C -]
...	...	...	...	...	...	...

REAL-LIFE EXAMPLE: WAREHOUSE INVENTORY SYSTEM



How It works:

Suppliers (Producers) add items into the warehouse storage (buffer). Stores (Consumers) take items from the storage when available. Semaphores ensure safe and efficient access even with multiple suppliers and stores working at the same time.

EXPLANATION:

Producers generate items and want to place them into the shared buffer. They must wait if the buffer is full (empty = 0). Consumers want to take items from the buffer. They must wait if the buffer is empty (full = 0). Mutex ensures that only one task (producer or consumer) accesses the buffer at a time, preventing race conditions. After producing or consuming, tasks update the buffer index and signal the appropriate semaphore. This method allows multiple producers and consumers to work concurrently without data loss or corruption.

KEY POINTS:

- empty counts available free slots in the buffer.
- full counts available items in the buffer.
- Mutex ensures mutual exclusion while accessing buffer.
- Producers wait when buffer is full.
- Consumers wait when buffer is empty.
- Circular buffer is used for efficient space utilization.
- Works safely with any number of producers and consumers.

ADVANTAGES:

- Prevents buffer overflow and underflow.
- Ensures data consistency and safety.
- Supports multiple producers and consumers.
- Maximizes concurrency and throughput.
- Simple and efficient synchronization.

ASSUMPTIONS:

- Buffer size (N) is fixed and greater than 0.
- All semaphore operations are atomic.
- Producers and consumers are independent tasks.
- IN and OUT are initialized to 0.
- Scheduler handles task execution.